

RetailPOS

Low Level Design Document

Microservices Architecture · Event-Driven · Multi-Tenant

Version 1.0 · May 2026

Technology Stack: .NET 8 · Angular 19 · RabbitMQ · SQL Server · Redis · SignalR · Docker

Table of Contents

1	System Overview & Purpose	3
2	Architecture Overview	3
3	Microservice Deep-Dives	4
3.1	Identity Service	4
3.2	Catalog Service	5
3.3	Orders Service	6
3.4	Payment Service	7
3.5	Returns Service	8
3.6	Admin Service	9
3.7	Notification Service	10
3.8	AI Service	11
3.9	API Gateway	11
4	Data Models & Entity Schemas	12
5	Event-Driven Messaging Contracts	14
6	Saga Orchestration Flows	15
7	Permission & RBAC Design	17
8	Infrastructure & Deployment	18
9	Frontend Architecture (Angular UI)	19
10	Cross-Cutting Concerns	20

1. System Overview & Purpose

RetailPOS is a cloud-ready, multi-tenant Point-of-Sale platform built on a microservices architecture. It enables retail chains to manage multiple stores from a single platform, with each store operating in full isolation through tenant-scoped data access. The system handles the complete retail lifecycle — from staff onboarding and product catalog management through billing, payment processing, returns, and AI-assisted operations.

Goal	Description
Multi-Tenancy	Every store operates in data isolation via StoreId scoping. A TenantProvider middleware injects the StoreId from JWT claims.
Event-Driven	Services communicate asynchronously via RabbitMQ (MassTransit). No direct service-to-service HTTP calls in business flows.
Saga Orchestration	Complex multi-step workflows (Checkout, Returns) are managed by MassTransit State Machines ensuring consistency.
RBAC Security	Fine-grained permissions (e.g., orders:create, admin:inventory:adjust) enforced at the API Gateway and service level.
Real-Time Notifications	SignalR WebSocket hub broadcasts store-scoped events (low stock alerts, return approvals) to connected clients.
AI Assistant	Gemini 1.5 Flash LLM integration provides store staff with live inventory Q&A; via a chat interface.

2. Architecture Overview

2.1 Service Topology

The platform follows a microservices pattern where each bounded context is an independent deployable unit. All client traffic is routed through an Ocelot API Gateway. Internal service communication uses RabbitMQ via MassTransit for events and commands. Each service owns its own database schema (same SQL Server instance, separate schemas).

Layer	Component	Technology	Port
Frontend	POS UI	Angular 19, TypeScript, SignalR Client	4200
Gateway	API Gateway	Ocelot, JWT Auth, Circuit Breaker, QoS	5000
Business	Identity Service	.NET 8, ASP.NET Core, EF Core, JWT	5001
Business	Catalog Service	.NET 8, EF Core, MassTransit	5002
Business	Orders Service	.NET 8, EF Core, MassTransit Saga	5003
Business	Admin Service	.NET 8, EF Core, MassTransit Saga	5004
Business	Payment Service	.NET 8, Razorpay SDK, MassTransit	5005
Business	Returns Service	.NET 8, EF Core, MassTransit Saga	5006
Business	Notification Service	.NET 8, SignalR Hub, MassTransit	5176
AI	AI Service	.NET 8, Gemini 1.5 Flash REST API	5007
Messaging	RabbitMQ	RabbitMQ 3-management, MassTransit	5672/15672
Database	SQL Server	MSSQL Server 2025, EF Core Migrations	1433
Cache	Redis	Redis Alpine (Token storage)	6379
Logging	Seq	Serilog + Seq structured logging	8081

2.2 Communication Patterns

Pattern	Used For	Technology
Sync HTTP (Client→Gateway→Service)	All user-initiated CRUD operations, queries, login/register	REST, Ocelot reverse proxy
Async Event Publishing	UserRegistered, ProductCreated, OrderPlaced, StockAdjusted	MassTransit Publish → RabbitMQ
Async Command Sending	DeductStock, RefundPayment, FinalizeOrder, RestockItem	MassTransit Send → specific consumer
Saga Orchestration	Checkout flow, Return approval/refund flow	MassTransit StateMachine with persisted state
WebSocket Push	Real-time notifications to store dashboards and POS terminals	SignalR Hub, store-scoped groups

3. Microservice Deep-Dives

3.1 Identity Service

Responsible for all authentication, authorization, and user management. It is the only service that issues JWT tokens and manages refresh token rotation. Supports email/password login with OTP-based 2FA and Google OAuth. Uses a multi-tenant RBAC model where a single user can have different roles in different stores.

Method	Endpoint	Auth	Description
POST	/api/auth/register	Public	Register new user; triggers email OTP verification
POST	/api/auth/verify-email	Public	Verify email with OTP code
POST	/api/auth/login	Public	Initiate login; sends login OTP to email
POST	/api/auth/verify-login-otp	Public	Verify OTP; returns JWT + RefreshToken
POST	/api/auth/refresh	Public	Exchange refresh token for new JWT
POST	/api/auth/google-login	Public	Google OAuth login/register
POST	/api/auth/logout	Bearer	Revoke refresh token; blacklist in Redis
GET	/api/auth/stores	Public	List available stores for store-selector UI

Key Design Decisions:

- JWT tokens carry StoreId, UserId, Email, and permission claims. Services extract StoreId via TenantProvider middleware — no inter-service calls needed for tenancy.
- Refresh tokens stored in Redis with TTL, enabling fast revocation without database queries on every request.
- User onboarding is saga-driven: UserRegisteredEvent triggers AdminService to create a StaffMember record, which the admin then assigns to a store.
- Passwords hashed using ASP.NET Core Identity PasswordHasher (PBKDF2-SHA256).

3.2 Catalog Service

Manages all product and category data scoped per store. Products have full lifecycle management including barcode/SKU, pricing (MRP vs. Selling Price), tax codes, stock levels, and reorder alerts. Consumes stock deduction and restock commands from the messaging bus to keep inventory consistent.

Method	Endpoint	Permission	Description
GET	/api/products	catalog:view	List all products for current store (tenant-scoped)
GET	/api/products/{id}	catalog:view	Get single product by ID
POST	/api/products	catalog:manage	Create product; publishes ProductCreatedEvent
PUT	/api/products/{id}	catalog:manage	Update product; publishes ProductUpdatedEvent
DELETE	/api/products/{id}	catalog:delete	Soft-delete product (IsActive = false)

GET	/api/categories	catalog:categories:view	List categories for current store
POST	/api/categories	catalog:categories:edit	Create category; publishes CategoryCreatedEvent
GET	/api/sync/products	all:all (internal)	Full product list for Admin sync consumer

Message Consumers:

- SagaDeductStockConsumer — reduces StockQuantity for each product in a finalized order; publishes StockDeductedEvent or StockDeductionFailedEvent.
- StockAdjustedConsumer — handles manual inventory adjustments from AdminService.
- OrderReturnedConsumer — restocks products when a return saga completes.
- CategoryConsumer — syncs categories pushed from AdminService to store's catalog.

3.3 Orders Service

Handles the complete billing lifecycle: Draft → Held → Finalized. A bill (order) has line items with product snapshot data. The Checkout Saga is hosted here and coordinates payment confirmation with stock deduction before finalizing the bill. Customer records are lazily created from mobile numbers.

Method	Endpoint	Permission	Description
POST	/api/bills	orders:create	Create draft bill; publishes CheckoutInitiatedEvent to start saga
GET	/api/bills	orders:view	List bills for current store (paginated)
GET	/api/bills/{id}	orders:view	Get bill with line items detail
PUT	/api/bills/{id}/hold	orders:hold	Put bill on hold (Status = 'Held')
POST	/api/customers	orders:create	Create/find customer by mobile number
GET	/api/customers	orders:view	Search customers by mobile

Checkout Saga States:

State	Trigger	Next Action
PaymentProcessing	CheckoutInitiatedEvent received	Wait for PaymentProcessedEvent; set 5-min timeout
WaitingForInitiation	PaymentProcessedEvent arrives before checkout event (race condition)	Wait for CheckoutInitiatedEvent; then proceed
StockDeductionProcessing	Payment confirmed as Success	Publish DeductStockCommand to CatalogService
Refunding	StockDeductionFailedEvent OR timeout	Publish RefundPaymentCommand to PaymentService
Completed (Final)	StockDeductedEvent received	Publish FinalizeOrderCommand; saga ends
Failed (Final)	Payment failed OR refund completed	Saga ends; bill remains unfinalized

3.4 Payment Service

Integrates with Razorpay for UPI/Card payments and handles cash payments natively. Receives commands from the Checkout Saga and emits payment status events. Supports refunds triggered by saga commands.

Method	Endpoint	Permission	Description
POST	/api/payments/create-order	payments:create_order	Create Razorpay order for UPI/Card checkout
POST	/api/payments/verify	payments:verify	Verify Razorpay payment signature; publishes PaymentProcessedEvent
POST	/api/payments/cash	payments:create_order	Record cash payment; directly publishes PaymentProcessedEvent
GET	/api/payments/{billId}	payments:verify	Get payment record for a bill

- Payment modes: Cash, Card, UPI. Razorpay integration used for Card/UPI with webhook-based signature verification.
- CheckoutSagaCommandsConsumer — listens for RefundPaymentCommand and processes refunds via Razorpay API, then publishes PaymentRefundedEvent.

3.5 Returns Service

Manages the full return lifecycle: a cashier initiates, a manager approves/rejects, and the Return Saga coordinates the refund and restock. Return states flow from Initiated → Approved/Rejected → Refunded.

Method	Endpoint	Permission	Description
POST	/api/returns	returns:initiate	Initiate return; publishes ReturnInitiatedEvent to start saga
GET	/api/returns	returns:view	List returns for current store
PUT	/api/returns/{id}/approve	returns:approve	Manager approves return; publishes ReturnApprovedEvent
PUT	/api/returns/{id}/reject	returns:approve	Manager rejects return; publishes ReturnRejectedEvent

Return Saga States:

State	Trigger	Action
AwaitingApproval	ReturnInitiatedEvent	Wait for manager decision
Refunding	ReturnApprovedEvent	Send RefundPaymentCommand to PaymentService
Restocking	PaymentRefundedEvent	Send RestockItemCommand to CatalogService
Completed (Final)	StockRestockedEvent	Publish FinalizeReturnCommand + OrderReturnedEvent; end saga
Rejected (Final)	ReturnRejectedEvent	Publish FinalizeReturnCommand(Rejected); end saga

3.6 Admin Service

Super-admin portal service for managing stores, staff assignments, inventory adjustments, and cross-store reporting. Maintains its own read-model (synced products, synced orders) built from events published by other services. Uses the Onboarding Saga for new user approval.

Controller	Key Endpoints	Permission
StoresController	GET/POST/PUT /api/stores — CRUD for store management	admin:stores:manage
StaffController	GET /api/staff — list pending staff; PUT /api/staff/{id}/assign — assign to store	users:manage
InventoryController	GET /api/inventory — list products; POST /api/inventory/adjust — stock adjustment	admin:inventory:adjust
ReportsController	GET /api/reports/sales, /api/reports/returns — aggregated analytics	admin:reports:view
DashboardController	GET /api/dashboard — KPIs: revenue, top products, store comparisons	admin:reports:view
CategoriesController	POST /api/categories — push categories to all stores via CategoryCreatedEvent	catalog:categories:edit

Onboarding Saga: When a new user registers, UserRegisteredEvent triggers the OnboardingStateMachine. It waits for admin assignment (StaffAssignedEvent) and then sends StaffAssignedEvent to IdentityService to grant store access.

3.7 Notification Service

Provides real-time in-app notifications via SignalR WebSockets. Consumes events from RabbitMQ, persists notifications in its own database, and pushes them to store-scoped SignalR groups. Supports read/unread status.

Event Consumed	Notification Content	Target Group
OrderPlacedEvent	'New order #X for ■Y placed'	Store_{StoreId}
PaymentProcessedEvent	'Payment of ■X received for Bill #Y'	Store_{StoreId}
ReturnInitiatedEvent	'Return request initiated for Bill #X'	Store_{StoreId}
ReturnApprovedEvent	'Return #X has been approved'	Store_{StoreId}
ProductCreatedEvent	'New product added to catalog'	Store_{StoreId}
StockAdjustedEvent	'Stock adjusted for product #X'	Store_{StoreId}

- SignalR Hub groups are store-scoped: clients call JoinStoreGroup(storeId) on connection.
- StoreId extracted from JWT claims automatically on WebSocket connection in OnConnectedAsync().

3.8 AI Service

A lightweight proxy service that integrates Google Gemini 1.5 Flash for natural-language store assistance. Before calling Gemini, it fetches live inventory data from the API Gateway and injects it as context into the system prompt. This enables queries like 'what is out of stock?' or 'show me the highest priced item'.

Aspect	Detail
Endpoint	POST /api/chat — accepts {message} and returns {response}
LLM	Google Gemini 1.5 Flash via REST API (GEMINI_API_KEY env var)
Context Injection	Fetches /gateway/catalog/categories and /gateway/catalog/products before each call; injects as markdown table in system prompt
Graceful Degradation	If API key missing or other services down, returns friendly fallback message
Auth forwarding	Passes Bearer token to internal gateway calls for tenant-scoped catalog data

3.9 API Gateway (Ocelot)

Single entry point for all client requests. Routes traffic to downstream services with JWT authentication, circuit-breaking, and QoS timeouts. Public routes (login, register) bypass auth; all business routes require Bearer token.

Gateway Prefix	Downstream Service	Auth Required	Timeout
/gateway/auth/login, register, verify-*	Identity Service :5001	No	5s
/gateway/auth/{everything}	Identity Service :5001	Yes	5s
/gateway/catalog/{everything}	Catalog Service :5002	Yes	5s
/gateway/orders/{everything}	Orders Service :5003	Yes	5s
/gateway/admin/{everything}	Admin Service :5004	Yes	5s
/gateway/payments/{everything}	Payment Service :5005	Yes	5s

/gateway/ai/{everything}	AI Service :5007	No	10s
--------------------------	------------------	----	-----

Circuit Breaker: Each route allows 3 exceptions before breaking; break duration 10 seconds.

4. Data Models & Entity Schemas

Each microservice owns its own database tables. The same SQL Server instance is shared, but schemas are logically separated. All entities carry a StoreId for multi-tenant isolation. EF Core Migrations manage schema evolution.

4.1 Identity Service — Entities

Entity	Field	Type	Notes
User	Id, Email, FullName, PasswordHash, EmployeeCode	int, string, string, string, string	Email unique, PK=Id
User (cont.)	Otp, OtpExpiry, IsEmailVerified, VerificationOtp, VerificationOtpExpiry	string?, DateTime?, bool, string?, DateTime?	OTP lifecycle fields
User (cont.)	RefreshToken, RefreshTokenExpiry	string?, DateTime?	Rotated on each use
Role	Id, Name	int, string(50)	e.g. Owner, Manager, Cashier
Permission	Id, Code, Description	int, string(100), string(250)	Code = 'orders:create'
RolePermission	RoleId, PermissionId	int, int	Many-to-many join
UserStoreRole	UserId, StoreId, RoleId	int, int, int	Composite PK — user can have different role per store
Store	Id, StoreCode, Name, Location, IsActive	int, string, string, string, bool	Synced from AdminService via event

4.2 Catalog Service — Entities

Entity	Field	Type	Notes
Product	Id, Sku, Barcode, Name, MRP, SellingPrice	int, str(50), str(50), str(200), decimal, decimal	MRP ≥ SellingPrice
Product (cont.)	TaxCode, ReorderLevel, IsActive, StockQuantity, CategoryId, StoreId	str(20), int, bool, int, int FK, int	StockQuantity updated via events
Category	Id, Name, Description, IsActive, StoreId	int, str(100), str(500), bool, int	Id manually set (no auto-increment)

4.3 Orders Service — Entities

Entity	Field	Type	Notes
Bill	Id, BillNumber, Date, StoreId, CashierId, CustomerMobile	int, str(50), DateTime, int, int, str(20)	Auto-generated BillNumber
Bill (cont.)	TotalAmount, TaxAmount, Status, CustomerName	decimal, decimal, str(20), string	Status: Draft Held Finalized
BillItem	Id, BillId FK, ProductId, ProductName, UnitPrice, Quantity, SubTotal, StoreId	int, int, int, str(200), decimal, int, decimal, int	Snapshot of product at sale time
Customer	Id, Mobile, Name, CreatedAt, StoreId	int, str(20) regex, str(100), DateTime, int	Mobile must be 10 digits

4.4 Payment / Returns / Admin — Entities

Service	Entity	Key Fields
PaymentService	Payment	Id, BillId, PaymentMode (Cash Card UPI), Amount, ReferenceNumber, StoreId
ReturnsService	Return	Id, OriginalBillId, ProductId, Quantity, RefundAmount, Reason, Status (Initiated Approved Refunded Rejected), ManagerApprovalNote, CustomerMobile, StoreId, Date
AdminService	AdminStoreEntity	Id, StoreCode, Name, Location, IsActive
AdminService	StaffMember	UserId (PK, no auto-inc), Email, FullName, IsAssigned, AssignedStoreId, AssignedRole, RegisteredDate
AdminService	SyncedProduct	Id, Name, Sku, StoreId — read-model from ProductCreatedEvent
NotificationService	Notification	Id, StoreId, Title, Message, IsRead, CreatedAt

5. Event-Driven Messaging Contracts

All inter-service messages are defined as interfaces in the shared RetailPOS.Contracts library. MassTransit uses these interfaces for type-safe publish/consume. Each event/command carries the minimum data needed by consumers, avoiding tight coupling.

Contract	Type	Publisher	Consumers	Key Fields
CheckoutInitiatedEvent	Event	OrdersService	CheckoutSaga (Orders)	CorrelationId, OrderId, StoreId, CashierId, TotalAmount, TaxAmount, CustomerMobile, Items[]
PaymentProcessedEvent	Event	PaymentService	CheckoutSaga, AdminService, NotificationService	CorrelationId, PaymentId, OrderId, Amount, PaymentMode, Status
DeductStockCommand	Command	CheckoutSaga	SagaDeductStockConsumer (Catalog)	CorrelationId, OrderId, Items[{ProductId, Quantity}]
StockDeductedEvent	Event	CatalogService	CheckoutSaga (Orders)	CorrelationId, OrderId
StockDeductionFailedEvent	Event	CatalogService	CheckoutSaga (Orders)	CorrelationId, OrderId, Reason
FinalizeOrderCommand	Command	CheckoutSaga	SagaOrderCommandsConsumer (Orders)	OrderId
RefundPaymentCommand	Command	CheckoutSaga / ReturnSaga	CheckoutSagaCommandsConsumer (Payment)	OrderId, ReturnId?, PaymentId, RefundAmount, Reason
PaymentRefundedEvent	Event	PaymentService	CheckoutSaga, ReturnSaga	OrderId, ReturnId?, PaymentId, RefundedAmount
ReturnInitiatedEvent	Event	ReturnsService	ReturnSaga, OrdersService, NotificationService	OrderId, ServiceReturnId, StoreId, TotalRefund, CustomerMobile, Items[]
ReturnApprovedEvent	Event	ReturnsService	ReturnSaga	ReturnId, Note
ReturnRejectedEvent	Event	ReturnsService	ReturnSaga	ReturnId, Reason
RestockItemCommand	Command	ReturnSaga	OrderReturnedConsumer (Catalog)	OrderId, Items[{ProductId, Quantity}]
OrderReturnedEvent	Event	ReturnSaga	OrderReturnedConsumer (Orders), AdminService	OrderId, ReturnId, StoreId, RefundAmount, CustomerMobile, Items[]
UserRegisteredEvent	Event	IdentityService	UserRegisteredConsumer (Admin), OnboardingSaga	UserId, Email, FullName, RoleName, StoreId?
StaffAssignedEvent	Event	AdminService	StaffAssignedConsumer (Identity)	UserId, StoreId, RoleName
ProductCreatedEvent	Event	CatalogService	ProductSyncConsumer (Admin), NotificationService	ProductId, Name, Sku, InitialStock, StoreId

ProductUpdatedEvent	Event	CatalogService	AdminService	ProductId, Name, Sku
CategoryCreatedEvent	Event	AdminService	CategoryConsumer (Catalog)	Id, Name, Description, StoreId
StockAdjustedEvent	Event	AdminService	StockAdjustedConsumer (Catalog), NotificationService	ProductId, Quantity (delta), StoreId, Reason

6. Saga Orchestration Flows

6.1 Checkout Saga — Step-by-Step Flow

Step	Actor	Action / Message	Saga State
1	Cashier (UI)	Clicks 'Pay' on the billing screen	—
2	OrdersService	Creates Draft Bill; publishes CheckoutInitiatedEvent	— (pre-saga)
3	Checkout Saga	Receives CheckoutInitiatedEvent; starts 5-min timer	PaymentProcessing
4	UI / PaymentService	Cashier processes payment (Cash/UPI/Card); PaymentService publishes PaymentProcessedEvent	PaymentProcessing
5a	Checkout Saga	[Payment SUCCESS] Publishes DeductStockCommand to CatalogService	StockDeductionProcessing
5b	Checkout Saga	[Payment FAILED] Saga finalizes in Failed state	Failed (end)
6a	CatalogService	[Stock OK] Publishes StockDeductedEvent	StockDeductionProcessing
6b	CatalogService	[Out of Stock] Publishes StockDeductionFailedEvent	Refunding
7a	Checkout Saga	[Stock OK] Publishes FinalizeOrderCommand; cancels timer	Completed (end)
7b	Checkout Saga	[Stock Failed] Publishes RefundPaymentCommand to PaymentService	Refunding
8b	PaymentService	Processes refund; publishes PaymentRefundedEvent	Refunding
9b	Checkout Saga	Saga ends in Failed state after refund confirmation	Failed (end)
Time out	Checkout Saga	If 5-min timeout fires: refund if payment processed, else just fail	Failed (end)

6.2 Return Saga — Step-by-Step Flow

Step	Actor	Action / Message	Saga State
1	Cashier	Initiates return request via POS UI (bill ID + items + reason)	—
2	ReturnsService	Creates Return record (Status=Initiated); publishes ReturnInitiatedEvent	— (pre-saga)
3	Return Saga	Receives ReturnInitiatedEvent; stores return context	AwaitingApproval
4a	Manager	Approves return via Admin/POS UI; publishes ReturnApprovedEvent	AwaitingApproval
4b	Manager	Rejects return; publishes ReturnRejectedEvent	AwaitingApproval
5a	Return Saga	[Approved] Publishes RefundPaymentCommand to PaymentService	Refunding
5b	Return Saga	[Rejected] Publishes FinalizeReturnCommand(Rejected); saga ends	Rejected (end)
6	PaymentService	Issues refund; publishes PaymentRefundedEvent	Refunding

7	Return Saga	Receives refund confirmation; publishes RestockItemCommand	Restocking
8	CatalogService	Restores product stock; publishes StockRestockedEvent	Restocking
9	Return Saga	Publishes FinalizeReturnCommand(Refunded) + OrderReturnedEvent; ends	Completed (end)

7. Permission & RBAC Design

The system uses attribute-based authorization via permission codes. Each API endpoint is decorated with `[RequirePermission("permission:code")]`. The gateway validates JWT authenticity; each service validates permission claims. Roles are pre-configured in the database with a set of permissions.

Permission Code	Description	Typical Role
all:all	Full superuser access	Owner (Super Admin)
orders:create	Create bills and initiate checkout	Cashier, Manager, Owner
orders:view	View orders for current store	Cashier, Manager, Owner
orders:view_all	View all orders across stores	Owner
orders:hold / orders:void	Put bill on hold or void it	Manager, Owner
returns:initiate	Initiate a return request	Cashier, Manager, Owner
returns:approve	Approve or reject return requests	Manager, Owner
payments:create_order	Create Razorpay payment order	Cashier, Manager, Owner
payments:verify	Verify and record payment	Cashier, Manager, Owner
catalog:view	Browse products and categories	All authenticated users
catalog:manage	Create/update products	Manager, Owner
catalog:delete	Soft-delete products	Owner
catalog:categories:edit	Create/modify categories	Manager, Owner
admin:inventory:adjust	Manual stock adjustment	Manager, Owner
admin:stores:manage	Create/update store configurations	Owner
admin:reports:view	View sales and returns analytics	Manager, Owner
users:manage	Assign/manage staff members	Owner
notifications:view	View and mark notifications	All authenticated users

8. Infrastructure & Deployment

The entire stack is containerized using Docker and orchestrated with Docker Compose. All services use health checks to ensure dependencies (DB, RabbitMQ) are ready before starting.

Component	Image / Technology	Purpose	Ports
SQL Server	mcr.microsoft.com/mssql/server:2025-latest	Primary data store for all services	1433
RabbitMQ	rabbitmq:3-management	Message broker for async events; management UI at :15672	5672 / 15672
Redis	redis:alpine	Refresh token storage, token blacklisting	6379
Seq	datalust/seq:2024.3	Centralized structured log aggregation (Serilog sink)	5341 / 8081
All Services	.NET 8 (Dockerfile per service)	Each service builds from its own Dockerfile; shares .env secrets	5001–5176
Angular UI	Node:LTS + Angular CLI	Served via ng serve in dev; proxied through API Gateway in prod	4200

8.1 Database Initialization Strategy

- Each service runs EF Core Migrations automatically on startup via DbInitializer.
- Seed data (roles, permissions, a default store, initial admin user) is seeded by IdentityService and CatalogService initializers.
- MassTransit Outbox Pattern is implemented in Admin and Catalog services (AddMassTransitOutbox migration) to ensure at-least-once delivery.

8.2 Service Dependency Order

Service	Depends On (Health Check Required)
identity-service, catalog-service, orders-service, admin-service, payment-service, returns-service, notification-service	db (healthy) + rabbitmq (started)
ai-service	rabbitmq (started) only — no DB dependency
api-gateway	All business services started
ui (Angular)	api-gateway started

9. Frontend Architecture (Angular 19 UI)

The POS UI is an Angular 19 SPA using standalone components, lazy-loaded feature modules, and reactive services. It connects to the API Gateway for all HTTP calls and to the NotificationService SignalR hub for real-time updates.

Module / Feature	Components	Description
Auth Feature	LoginComponent, SignupComponent, OtpVerificationComponent, PendingApprovalComponent	Login with OTP 2FA, registration, email verification, post-register pending state
POS Feature	BillingComponent, PaymentComponent, PaymentDialogComponent, ReturnsComponent	Main cashier screens: product search, cart, payment dialog (cash/UPI/card), return initiation
Admin Feature	DashboardComponent, InventoryComponent, StoresComponent, StaffComponent, CategoriesComponent, TransactionsComponent	Store management, staff assignment, inventory adjustment, analytics dashboard
Marketing Feature	LandingComponent	Public landing/marketing page for the SaaS product
Shared Components	NavbarComponent, SidebarComponent, AIChatbotComponent	Global layout, navigation, and AI chat bubble overlay
Core Services	AuthService, CatalogApiService, OrderApiService, AdminApiService, PaymentApiService, AiApiService, NotificationService, SignalRService, ThemeService	API wrappers per bounded context; SignalR connection manager; dark/light theme
Guards & Interceptors	AuthGuard, AuthInterceptor, ErrorInterceptor	Route protection, automatic Bearer token injection, global error handling

Key Frontend Patterns:

- AuthInterceptor automatically attaches the JWT Bearer token to all outgoing HTTP requests.
- SignalRService connects to NotificationService hub on login; auto-joins the user's store group.
- The AI Chatbot is a floating overlay component available on all authenticated pages.
- Route guards prevent unauthenticated access; lazy loading keeps initial bundle small.

10. Cross-Cutting Concerns

Concern	Implementation	Location
Structured Logging	Serilog with Seq sink; all services emit structured JSON logs with CorrelationId for distributed tracing	RetailPOS.Common / SerilogConfiguration.cs
Correlation ID	CorrelationIdMiddleware generates/propagates X-Correlation-ID header across all requests for end-to-end tracing	RetailPOS.Common / Middleware
Global Exception Handling	ExceptionMiddleware in each service catches DomainException, NotFoundException, BusinessException and maps to appropriate HTTP status codes	Each service / Middleware
Multi-Tenancy	ITenantProvider interface implemented by TenantProvider; extracts StoreId from JWT claims and exposes to repositories	Each service / Services/TenantProvider.cs
Generic Repository	IGenericRepository with CRUD + pagination; concrete implementations per service extend this base	Each service / Repositories/GenericRepository.cs
MassTransit Outbox	Transactional outbox pattern ensures events are not lost even if the broker is temporarily unavailable; persisted in SQL	AdminService, CatalogService
Circuit Breaking	Ocelot QoS: 3 exceptions before breaking, 10s break duration, 5–10s request timeout per service	ApiGateway / ocelot.json
Unit Testing	xUnit test projects per service covering service layer logic (AuthServiceTests, ProductServiceTests, BillServiceTests, PaymentServiceTests, NotificationServiceTests, ReturnServiceTests, DashboardServiceTests)	Tests/ directory
Health Checks	SQL Server and RabbitMQ health checks defined in docker-compose.yml; services start only after dependencies pass	docker-compose.yml
Environment Config	All secrets (DB connection, JWT key, Razorpay keys, Gemini API key, SMTP) loaded from .env file via env_file in Docker Compose	docker-compose.yml / appsettings.json

This document covers the complete Low Level Design of the RetailPOS platform. For High Level Design, ER Diagram, and Use Case diagrams refer to the /docs folder in the repository.